Python, environments & NumPy

Daily commands | Snippets share imports within each page; replace filenames and column names.

Environment and package checks

```
python -m venv .venv
# Linux/macOS activation:
source .venv/bin/activate
# Windows PowerShell activation:
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pip show scikit-learn
python -m pip check
python -m pip freeze > installed.txt
python -m pytest -q
python -m ruff check .
python -m ruff format .
```

Install/test commands require the named tools. Use your approved package source. pip freeze is an environment snapshot, not a hash-verified release lock.

Offline install from an approved bundle

```
python -m pip install --no-index \
  --find-links wheelhouse \
  --require-hashes -r requirements.lock
```

Requires a complete, approved lock with exact versions/hashes and compatible wheels. Backslash continuations above are for a Unix shell; use one line in PowerShell.

NumPy: arrays, axes and vectorization

```
import numpy as np
rng = np.random.default_rng(42)
X = rng.normal(size=(100, 4))
x = np.array([1, 2, np.nan], dtype=float)
X.shape, X.ndim, X.dtype
X[:, 0]                  # first column
X.mean(axis=0)           # column means
X.mean(axis=1)           # row means
np.nanmean(x)            # ignore NaN
np.isfinite(X).all()
np.clip(X, -3, 3)
np.where(X > 0, X, 0)
np.quantile(X, [0.25, 0.5, 0.75])
X @ X.T                 # matrix product
X.reshape(-1, 1)         # one column
X.astype(np.float32)     # less memory
```

Prefer vectorized operations over row loops. Broadcasting follows trailing dimensions. Check shape, units, missing values and memory before fitting.

Core Python patterns

```
from collections import Counter
from pathlib import Path
import json

items = ["a", "b", "a"]
counts = Counter(items)
unique = list(dict.fromkeys(items))
for i, item in enumerate(items):
    print(i, item)
pairs = dict(zip(["x", "y"], [1, 2]))
value = pairs.get("missing", 0)
squares = [x*x for x in range(5)]
```

Files and JSON

```
root = Path("data")
files = sorted(root.glob("*.csv"))
Path("outputs").mkdir(exist_ok=True)
cfg = json.loads(
    Path("config.json").read_text(
        encoding="utf-8"))
Path("outputs/run.json").write_text(
    json.dumps({"seed": 42}, indent=2),
    encoding="utf-8")
```

Use context managers for open files/connections. Never embed secrets or sensitive raw records in configs, logs or Git.

Timing, logging and debugging

```
import logging, time
logging.basicConfig(level=logging.INFO)
log = logging.getLogger(__name__)
t0 = time.perf_counter()
# ... run the operation ...
log.info("seconds=%.3f",
         time.perf_counter() - t0)
# breakpoint()  # interactive debugger
```

Inspect type(obj), shape/dtype, a small input sample and the full traceback. Catch specific exceptions; avoid silent except: pass.

Statistics: common first checks

```
from scipy import stats
a = np.array([1., 2., 3., 4.])
b = np.array([2., 3., 4., 6.])
stats.ttest_ind(a, b, equal_var=False)
stats.spearmanr(a, b)
stats.zscore(a, ddof=1)
np.std(a, ddof=1)       # sample SD
```

Welch's t-test compares independent group means under its assumptions. Report effect size and uncertainty; correlation is not causation. Account for repeated testing.

Primary docs: Python | NumPy | SciPy stats | pip

pandas, data checks & visualization

Import once: import pandas as pd; import numpy as np | df is your DataFrame.

Load, inspect and export

```
df = pd.read_csv("data.csv")
df = pd.read_parquet("data.parquet")
df.head(5)
df.shape
df.info()
df.describe(include="all")
df.isna().mean().sort_values(
    ascending=False)
df.nunique(dropna=False)
df["label"].value_counts(dropna=False)
df.duplicated().sum()
df.to_parquet("clean.parquet", index=False)
```

CSV/Parquet reads above are alternatives. Parquet needs an engine such as pyarrow. Check row count, keys, units, types, missingness and label balance.

Types, dates and text

```
df["score"] = pd.to_numeric(
    df["score"], errors="coerce")
df["date"] = pd.to_datetime(
    df["date"], errors="coerce", utc=True)
df["text"] = (
    df["text"].astype("string")
    .str.strip().str.lower())
df["year"] = df["date"].dt.year
df = df.dropna(subset=["label"])
df = df.drop_duplicates(subset=["id"])
```

Check the business meaning before dropping rows/duplicates. For ML, learn imputation, scaling and feature selection inside training folds (page 3).

Group, join and reshape

```
summary = df.groupby(
    "dept", dropna=False).agg(
        n=("id", "size"),
        avg=("score", "mean"))
joined = df.merge(
    lookup, on="id", how="left",
    validate="many_to_one", indicator=True)
wide = df.pivot_table(
    index="id", columns="dept",
    values="score", aggfunc="mean")
long = df.melt(
    id_vars=["id"], value_vars=["x", "y"])
combined = pd.concat(
    [part1, part2], ignore_index=True)
```

lookup, part1 and part2 are other DataFrames. Join validation catches accidental row multiplication; inspect unmatched rows in _merge.

Select, filter and update

```
df[["id", "score"]]
df.iloc[:10, :3]          # positions
df.loc[df["score"] > 0.8, ["id", "score"]]
mask = ((df["score"] > 0.8)
        & df["dept"].isin(["A", "B"]))
subset = df.loc[mask].copy()
df.loc[mask, "flag"] = True
df.sort_values("score", ascending=False)
df.nlargest(10, "score")
```

Use & / | with parentheses for elementwise filters. Use .loc for assignment; avoid chained assignment.

Time series and larger files

```
weekly = (df.set_index("date")["score"]
          .resample("W").mean())
for chunk in pd.read_csv(
        "large.csv", chunksize=100_000):
    # Replace with per-chunk processing.
    print(len(chunk))
df.memory_usage(deep=True).sum()
```

Use selected columns and explicit dtypes when loading. Try DuckDB/Polars when data exceeds a comfortable pandas working set.

Parameterized SQL read

```
import sqlite3
from contextlib import closing
with closing(sqlite3.connect(
    "file:data.db?mode=ro", uri=True)) as con:
    result = pd.read_sql_query(
        "SELECT * FROM events WHERE score > ?",
        con, params=(0.8,))
```

This example requires an existing SQLite database/table. Parameter syntax differs by driver. Do not concatenate user values into SQL.

Plot distributions and relationships

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.histplot(data=df, x="score", bins=30)
plt.tight_layout()
plt.savefig("scores.png", dpi=200)
plt.close()
sns.scatterplot(
    data=df, x="x", y="y", hue="label")
plt.show()
```

Inspect outliers, missingness, imbalance, time patterns and subgroup differences. Plot residuals for regression; label axes and units.

Primary docs: pandas basics | Groupby | Merge | Seaborn

A reliable scikit-learn workflow

One linked example: df has age, income, region and a binary target encoded as 0/1.

Split first; preprocess inside a pipeline

```
from sklearn.model_selection import (
    train_test_split, StratifiedKFold,
    cross_val_score, GridSearchCV)
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    StandardScaler, OneHotEncoder)
from sklearn.linear_model import (
    LogisticRegression)

X = df[["age", "income", "region"]]
y = df["target"]
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.2,
    stratify=y, random_state=42)

numeric = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler())
categorical = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore"))

prep = ColumnTransformer([
    ("num", numeric, ["age", "income"]),
    ("cat", categorical, ["region"])
])
model = make_pipeline(
    prep, LogisticRegression(max_iter=2000))

cv = StratifiedKFold(
    n_splits=5, shuffle=True, random_state=42)
auc = cross_val_score(
    model, Xtr, ytr, cv=cv,
    scoring="roc_auc", n_jobs=2)
print(auc.mean(), auc.std())
```

Use enough examples of each class for the chosen folds. Split by person/account with grouped CV when records are related; use forward/time splits for forecasting.

Leakage checks before every experiment

Exclude IDs, future information and post-outcome fields. Fit transformations on training data only. Keep related records together. Tune on training/validation data; reserve the final test set.

Useful model-selection alternatives

DummyClassifier / DummyRegressor: baseline.
GroupKFold: related entities.
TimeSeriesSplit: chronological evaluation.
RandomizedSearchCV: larger search spaces.
permutation_importance: held-out feature checks.

Tune on training folds, then test once

```
search = GridSearchCV(
    model,
    {"logisticregression__C": [0.1, 1, 10]},
    cv=cv, scoring="roc_auc", n_jobs=2)
search.fit(Xtr, ytr)
best = search.best_estimator_
pred = best.predict(Xte)
prob = best.predict_proba(Xte)[:, 1]
```

C is inverse regularization strength for this model. Pick any decision threshold using validation data, not the final test set.

Classification metrics

```
from sklearn.metrics import (
    classification_report, confusion_matrix,
    roc_auc_score, average_precision_score)
print(classification_report(
    yte, pred, zero_division=0))
print(confusion_matrix(yte, pred))
print(roc_auc_score(yte, prob))
print(average_precision_score(yte, prob))
```

Precision = TP/(TP+FP); recall = TP/(TP+FN). F1 balances both. Inspect rare-class performance. ROC-AUC uses scores; average precision summarizes the PR curve.

Regression metrics: separate example

```
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error,
    r2_score)
mae = mean_absolute_error(y_true, y_pred)
rmse = mean_squared_error(
    y_true, y_pred) ** 0.5
r2 = r2_score(y_true, y_pred)
```

MAE is in target units; RMSE penalizes large misses more. R-squared can be negative. MAPE is unstable near zero. Compare against a mean/median or seasonal baseline.

Save the whole fitted pipeline

```
import joblib
joblib.dump(best, "pipeline.joblib")
loaded = joblib.load("pipeline.joblib")
loaded.predict(Xte.head())
```

Load only trusted artifacts: pickle/joblib can execute code. Save data/schema, code, dependency and model versions. Cross-validation SD is not a confidence interval.

Primary docs: Pipelines | Splits | Metrics | Leakage

NLP, LLMs, PyTorch & documents

Model examples need approved local artifacts/dependencies. Compare results on your own task.

Text-classification baseline

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import (
    TfidfVectorizer)
from sklearn.linear_model import (
    LogisticRegression)
text_model = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2)),
    LogisticRegression(max_iter=2000))
text_model.fit(train_texts, y_train)
labels = text_model.predict(test_texts)
```

Often a useful baseline before fine-tuning an LLM. Keep vectorizer fitting inside cross-validation.

Embeddings and small-corpus retrieval

```
from sentence_transformers import (
    SentenceTransformer)
import numpy as np
encoder = SentenceTransformer(
    "/models/all-MiniLM-L6-v2",
    local_files_only=True)
docs = ["approved text A", "approved text B"]
D = encoder.encode(
    docs, normalize_embeddings=True)
q = encoder.encode(
    ["search question"],
    normalize_embeddings=True)[0]
scores = D @ q
top = np.argsort(scores)[::-1][:5]
hits = [(docs[i], scores[i]) for i in top]
```

Normalized dot product equals cosine similarity. This is exhaustive search for a small corpus. In RAG, filter by current user access before retrieval; record source citations.

Calling an internal model service

```
import requests
# endpoint/payload/headers follow YOUR API.
response = requests.post(
    endpoint, json=payload, headers=headers,
    timeout=(5, 60))
response.raise_for_status()
result = response.json()
```

Use the approved service schema, authentication and internal CA. Never disable TLS verification. Validate output structure; bound tokens, retries and concurrency.

LLM reminders

Temperature is not a correctness setting. Low temperature does not guarantee determinism. Respect model context limits and chat templates. Test factuality, citations and failure cases; treat retrieved instructions and generated code as untrusted.

PyTorch: one training epoch + inference

```
import torch
from torch import nn
device = "cuda" if torch.cuda.is_available() \
         else "cpu"
net = net.to(device)
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.AdamW(
    net.parameters(), lr=1e-3)
net.train()
for xb, yb in loader:
    xb, yb = xb.to(device), yb.to(device)
    opt.zero_grad(set_to_none=True)
    logits = net(xb)
    loss = loss_fn(logits, yb)
    loss.backward()
    opt.step()
net.eval()
with torch.inference_mode():
    logits = net(x_test.to(device))
    labels = logits.argmax(dim=1)
```

net, loader and x_test are your model/data. For class-index CrossEntropyLoss: raw logits [batch, classes]; yb is int64 [batch]. eval() changes dropout/batchnorm; inference_mode() disables gradient tracking.

Speech transcription with faster-whisper

```
from faster_whisper import WhisperModel
asr = WhisperModel(
    "/models/whisper-small-ct2",
    device="cpu", compute_type="int8")
segments, info = asr.transcribe(
    "audio.wav", vad_filter=True)
text = " ".join(s.text for s in segments)
```

Use a compatible local CTranslate2 model. Iterating segments executes transcription. Diarization is separate; measure WER against references and DER for speakers.

PDF text and image OCR

```
import pymupdf
with pymupdf.open("report.pdf") as pdf:
    text = "\n".join(
        page.get_text() for page in pdf)

import pytesseract
from PIL import Image
text = pytesseract.image_to_string(
    Image.open("scan.png"), lang="eng")
```

Install PyMuPDF for the pymupdf import. Image-only PDFs need rendering/OCR. pytesseract needs the Tesseract executable and language data installed.

Primary docs: PyTorch | Embeddings | Whisper | PDF | OCR

Choose tools & remember the basics

Practical starting points, not a leaderboard. Benchmark on representative data; check licenses and hardware.

Task	Common starting choices	Remember
Tables: classification / regression	scikit-learn: logistic regression / Ridge; random forests	Then benchmark XGBoost, LightGBM or CatBoost. Use a leak-free split.
Many categorical columns	CatBoost; or one-hot encoding + sklearn	Decide how to handle missing/unseen categories; compare to a simple baseline.
Time-series forecasting	Seasonal naive; statsmodels ETS / SARIMAX	Use rolling-origin evaluation; respect seasonality and future-data boundaries.
Clustering / anomaly detection	StandardScaler + KMeans/DBSCAN; IsolationForest	Scale distance-based inputs. Clusters/outlier scores do not establish ground truth.
Text labels / entities	TF-IDF + linear model; spaCy; Transformers	Use task-specific labels and error analysis; not every text task needs generation.
English semantic search	Sentence Transformers: all-MiniLM-L6-v2	Compact starting point for similarity; evaluate domain vocabulary and chunk length.
Multilingual retrieval	Sentence Transformers: multilingual-e5-base	Follow model-card query:/passage: prefixes and normalization rules.
Generation / summarization / RAG	Approved instruct LLM, e.g. Qwen3-8B	Validate runtime/model compatibility, grounding, access, context and resource use.
PDF / scanned documents	PyMuPDF or pdfplumber; Tesseract / PaddleOCR	Extract existing text first; OCR scans; inspect layout and table errors.
Speech / speaker separation	faster-whisper + Whisper; pyannote.audio for speakers	Choose model size by measured quality/latency; ASR and diarization need separate evaluation.
Images / computer vision	PyTorch + torchvision ResNet / EfficientNet	Start with transfer learning; match preprocessing to weights; preserve label mapping.
SQL / larger local analytics	DuckDB, Polars, PyArrow; pandas for smaller data	Select needed columns, use Parquet, and measure memory before adding distribution.
Tuning / explanation / tracking	Optuna; SHAP; MLflow	Budget tuning; explanations are not causality; log versions and sanitize traces.
APIs / validation / UI / checks	FastAPI, Pydantic, Streamlit; pytest, Ruff	Validate schemas and permissions; UI hiding is not authorization.

A short release / reproducibility check

```
git switch -c feature/small-change
git diff
git diff --staged
python -m pytest -q
```

Record code commit, data snapshot, split/seed, config, model and dependency versions. Keep README, Claude.md and relevant tests current.

Debug a bad result in this order

1. Check rows, keys, types, units and target labels.
2. Check leakage, splits and preprocessing.
3. Compare a simple baseline and failure slices.
4. Check metrics, thresholds and label quality.
5. Check resource limits and version changes.

Drift is a signal to investigate, not proof of failure.

Primary docs: sklearn | XGBoost | Model cards